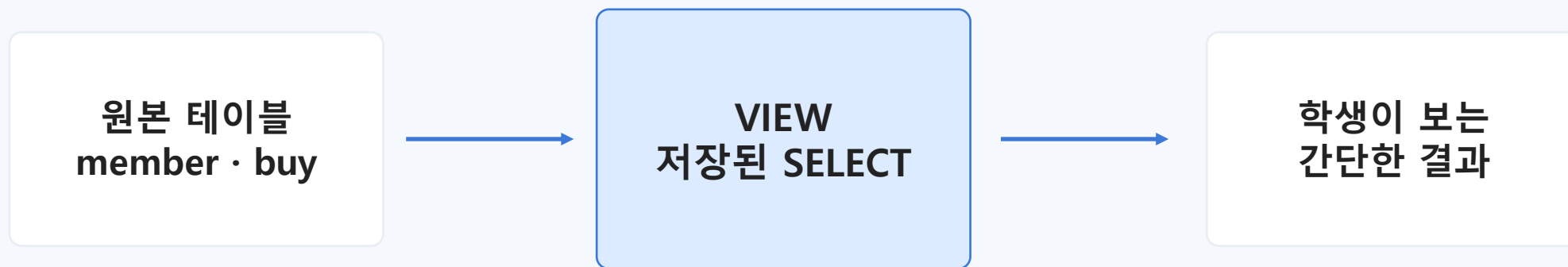


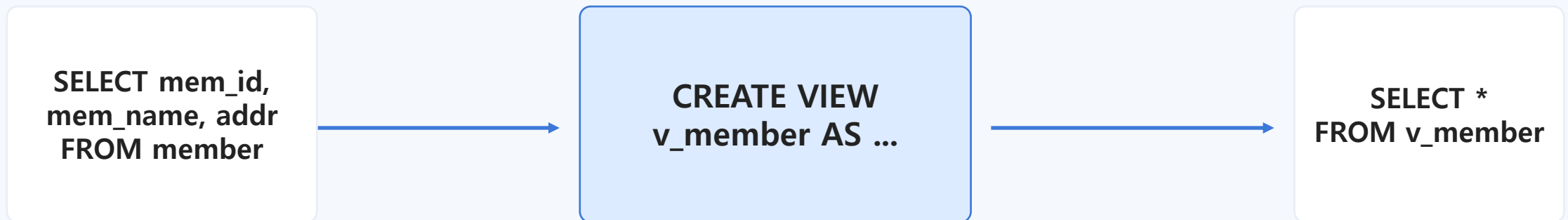
SQL View

복잡한 조회를 "이름 붙은 창"으로 바라보기



“뷰는 SELECT 문에 이름을 붙여 테이블처럼 쓰는 가상 테이블이다.”

데이터를 새로 복사해서 저장하는 것이 아니라, 원본 테이블을 다시 조회한다.



“긴 질문”을 저장해 두고, 나중에는 “짧은 이름”으로 부르는 방식

2. 왜 뷰를 사용할까?

뷰는 SQL을 잘 모르는 사용자에게도 복잡한 데이터 구조를 쉽게 보여 주는 도구입니다.



단순화

JOIN, CONCAT, GROUP BY
같은 복잡한 문장을 짧게 사
용



재사용

자주 쓰는 조회 기준을 한 번
정의해 반복 사용



일관성

모든 사람이 같은 계산식과
같은 열 이름을 사용



보안/제한

보여 줄 열과 행을 제한해 필
요한 정보만 제공

핵심: 뷰는 “데이터를 쉽게 보여 주는 약속된 창”이다.

3. 테이블과 뷰의 차이

테이블은 데이터를 직접 담고, 뷰는 데이터를 보는 방법을 담습니다.

테이블(Table)

실제 데이터 저장
INSERT로 행이 저장됨
원본 데이터의 중심
삭제하면 데이터도 사라짐

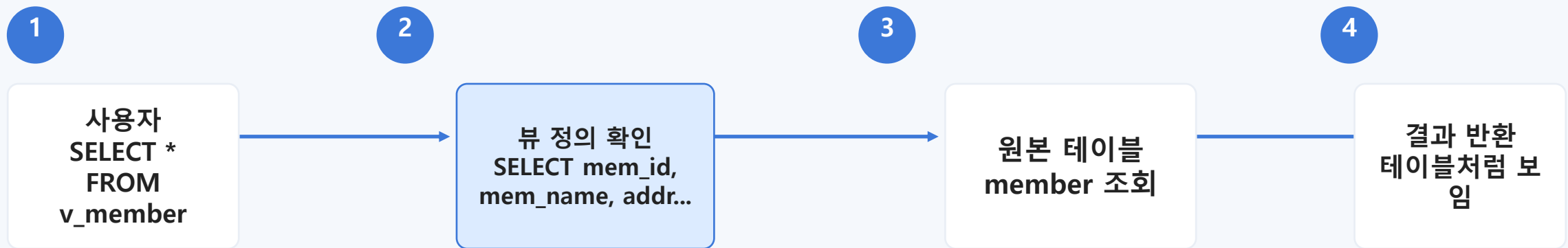
저장소

뷰(View)

SELECT 문을 저장
조회 시 원본 테이블을 참조
필요한 형태로 보여 주는 창
원본이 없으면 정상 작동 어려움

창 / 렌즈

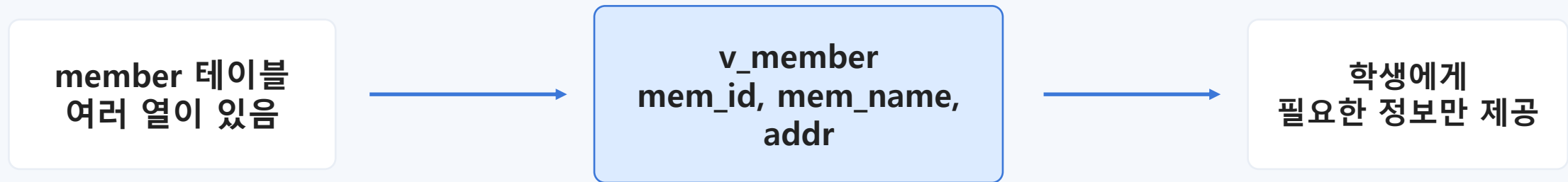
사용자가 뷰를 조회하면 DBMS는 뷰 안에 저장된 SELECT 문을 꺼내 원본 테이블을 조회합니다.



중요: 뷰 자체가 데이터를 복사해 가지고 있는 것이 아니다.

5. 첫 번째 뷰: 필요한 열만 보여 주기

member 테이블에는 여러 열이 있지만, 수업에서는 회원 아이디·이름·지역만 보여 주고 싶을 수 있습니다.



```
CREATE VIEW v_member AS  
  SELECT mem_id, mem_name, addr FROM member;  
  
SELECT * FROM v_member;
```

뷰는 “보여 줄 열을 정하는 창”으로 사용할 수 있다.

6. 뷰도 SELECT에서는 테이블처럼 사용한다

한 번 만든 뷰는 FROM 절에 테이블 이름처럼 사용할 수 있습니다.

```
SELECT mem_name, addr  
FROM v_member  
WHERE addr IN ('서울', '경기');
```

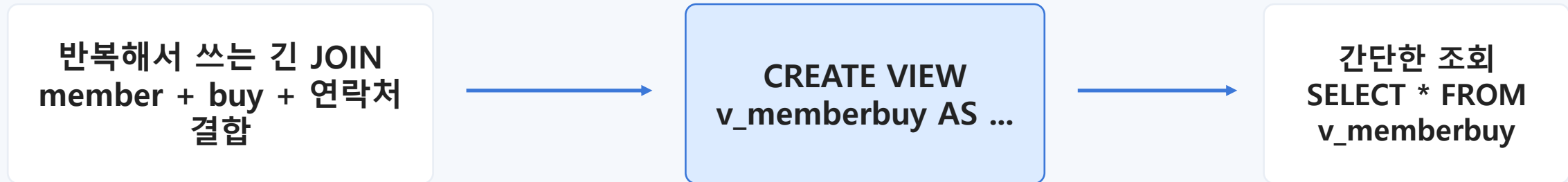
SELECT 대상이
member가 아니
라
v_member

하지만 실제 조회
는
member 테이블
에서 수행

학생 관점: 뷰를 테이블처럼 조회하면 된다.
DBMS 관점: 뷰의 SELECT 문을 원본 테이블에 적용한다.
WHERE 조건도 뷰 위에 추가로 걸 수 있다.

7. 복잡한 JOIN을 간단한 이름으로 만들기

긴 JOIN 문을 뷰로 저장하면 학생들은 조회 목적에 집중할 수 있습니다.

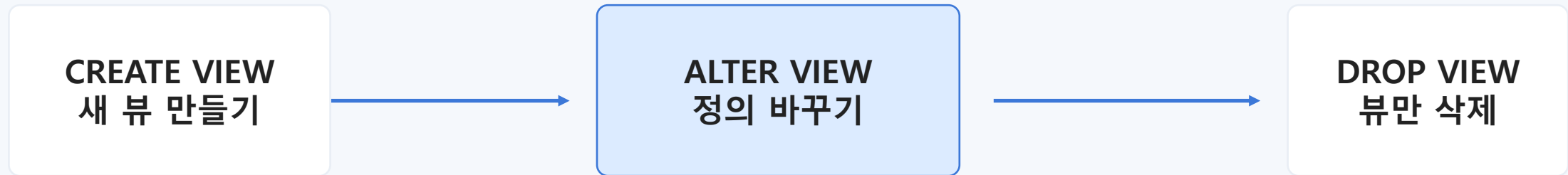


```
CREATE VIEW v_memberbuy AS
  SELECT B.mem_id, M.mem_name, B.prod_name, M.addr,
         CONCAT(M.phone1, M.phone2) '연락처'
  FROM buy B INNER JOIN member M ON B.mem_id = M.mem_id;

SELECT * FROM v_memberbuy WHERE mem_name = '블랙핑크';
```


8. 뷰의 생성·수정·삭제는 “정의”를 관리하는 일

뷰 관리는 원본 데이터를 바꾸는 것이 아니라, 뷰가 어떤 SELECT 문을 사용할지 바꾸는 것입니다.



```
ALTER VIEW v_viewtest1 AS ...  
DROP VIEW v_viewtest1;  
CREATE OR REPLACE VIEW v_viewtest2 AS ...
```

DROP VIEW는 뷰를 삭제할 뿐, 원본 member · buy 테이블의 데이터는 삭제하지 않는다.

9. 뷰의 정보를 확인하는 방법

뷰도 구조와 생성 SQL을 확인할 수 있습니다. 특히 “이 뷰가 어디서 왔는지” 확인할 때 중요합니다.

DESCRIBE v_viewtest2

뷰의 열 이름과 자료형 확인

SHOW CREATE VIEW v_viewtest2

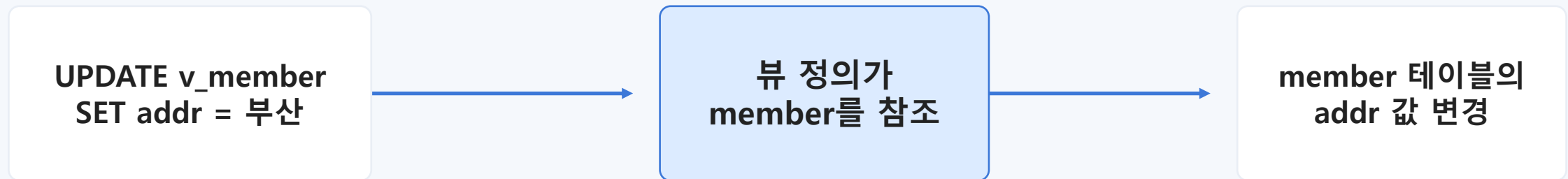
뷰가 어떤 SELECT 문으로
만들어졌는지 확인

```
DESCRIBE v_viewtest2;  
DESCRIBE member;  
SHOW CREATE VIEW v_viewtest2;
```

학생 질문: “이 뷰는 어떤 테이블을 보고 있나요?” → SHOW CREATE VIEW로 확인

10. 뷰를 통해 데이터를 수정할 수 있을까?

단순한 뷰는 UPDATE, DELETE, INSERT가 원본 테이블로 전달될 수 있습니다. 하지만 모든 뷰가 수정 가능한 것은 아닙니다.

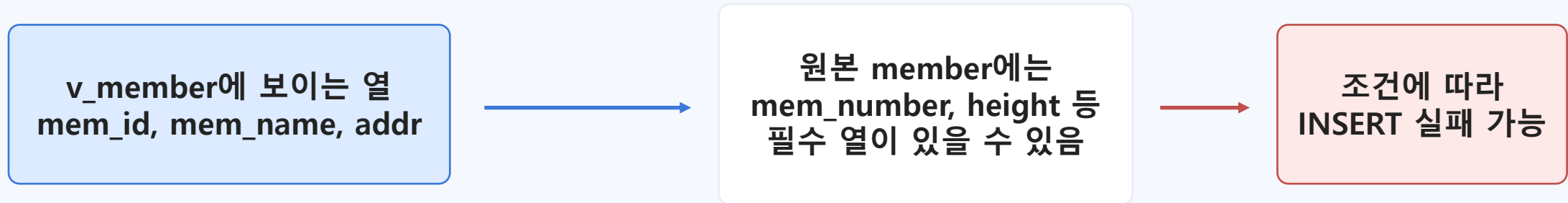


```
UPDATE v_member SET addr = '부산' WHERE mem_id='BLK';
```

표현을 정확히: “뷰 데이터를 바꾼다”보다 “뷰를 통해 원본 테이블을 바꾼다”가 맞다.

11. 뷰를 통한 입력은 왜 실패할 수 있을까?

뷰에는 일부 열만 보이지만, 원본 테이블에는 반드시 필요한 열이 더 있을 수 있습니다.



```
INSERT INTO v_member(mem_id, mem_name, addr)
VALUES('BTS', '방탄소년단', '경기');
```

뷰 입력의 성공 여부는 “뷰”가 아니라 “원본 테이블의 제약 조건”이 결정한다.

12. 조건이 있는 뷰: 보이는 데이터와 저장된 데이터가 다를 수 있다

v_height167은 height >= 167인 회원만 보여 주는 창입니다.



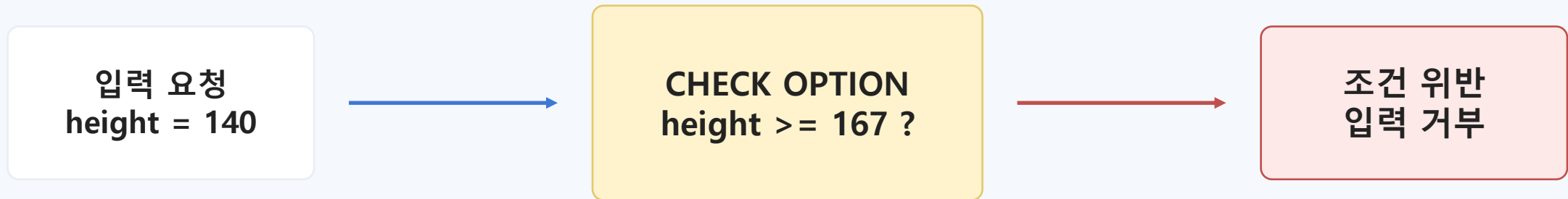
```
CREATE VIEW v_height167 AS
  SELECT * FROM member WHERE height >= 167;

INSERT INTO v_height167 VALUES('TRA','티아라',6,'서울',NULL,NULL,159,'2005-01-01');
```

학생에게 던질 질문: “입력했는데 SELECT * FROM v_height167에서 왜 안 보일까?”

13. WITH CHECK OPTION: 뷰 조건을 지키게 하는 문

WITH CHECK OPTION은 뷰의 WHERE 조건을 어기는 입력과 수정을 막는 안전장치입니다.

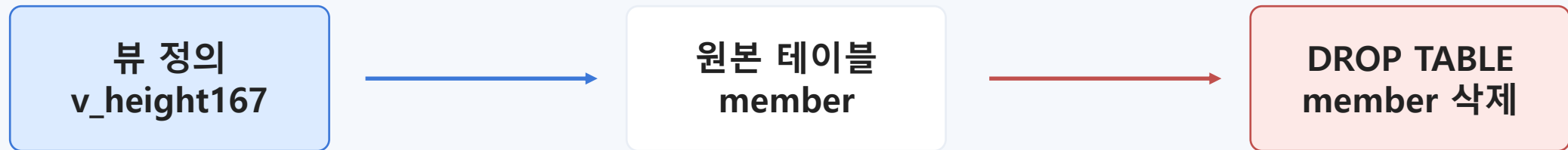


```
ALTER VIEW v_height167 AS  
  SELECT * FROM member WHERE height >= 167  
  WITH CHECK OPTION;
```

목적: “뷰에서 볼 수 없는 데이터”가 뷰를 통해 들어오는 문제를 막는다.

14. 원본 테이블이 삭제되면 뷰는 어떻게 될까?

뷰는 원본 테이블에 의존합니다. 참조하던 테이블이 사라지면 뷰 정의는 남아도 정상 조회가 어려워집니다.



```
DROP TABLE IF EXISTS buy, member;
```

```
SELECT * FROM v_height167;  
CHECK TABLE v_height167;
```

정리: 뷰는 “독립된 데이터 저장소”가 아니라 “원본 테이블을 보는 창”이다.

“뷰는 데이터를 저장하는 테이블이 아니라, 데이터를 바라보는 방법이다.”

그래서 뷰를 잘 쓰면 SQL은 짧아지고, 데이터 사용 기준은 더 분명해진다.

1
가상 테이블

2
복잡한 SQL 단순
화

3
원본 테이블 의존

4
수정 가능 조건
주의

v_member	기본 뷰 생성과 조회	필요한 열만 보여 주기
v_memberbuy	JOIN 뷰	복잡한 SQL 단순화
v_viewtest1/2	ALTER, DROP, DESCRIBE	뷰 정의 관리와 정보 확인
v_height167	WHERE 조건 뷰	보이는 데이터와 원본 데이터의 차이
WITH CHECK OPTION	조건 위반 차단	뷰 조건을 지키는 입력/수정
DROP TABLE + CHECK TABLE	원본 테이블 삭제	뷰가 원본에 의존함

필요하면 이전 코드 중심 슬라이드는 “실습 부록”으로만 사용하고, 본 자료는 개념 설명용으로 사용하세요.

개념 설명 뒤, 아래 정도의 짧은 코드만 먼저 실행하면 학생들이 뷰의 핵심을 이해하기 쉽습니다.

```
USE market_db;

-- 1. 기본 뷰
CREATE VIEW v_member AS
    SELECT mem_id, mem_name, addr FROM member;

SELECT * FROM v_member;

-- 2. 복잡한 SQL 단순화
CREATE VIEW v_memberbuy AS
    SELECT B.mem_id, M.mem_name, B.prod_name, M.addr,
           CONCAT(M.phone1, M.phone2) '연락처'
    FROM buy B INNER JOIN member M ON B.mem_id = M.mem_id;

SELECT * FROM v_memberbuy WHERE mem_name = '블랙핑크';

-- 3. 정보 확인
DESCRIBE v_member;
SHOW CREATE VIEW v_member;
```